JAILEDDOTFUN
Security Review



DECEMBER, 2025

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Commit Hash
  - Scope
  - Roles
- Executive Summary
  - Issues Found
  - Security Grade
- Findings
  - High
  - Medium
  - Low
  - Informational
  - Gas

Protocol Summary

A Solana-based pickaxe training and exploration game. It blends NFT-style assets, STEPN-like exploration loops, and a Token-2022 economy with JAIL. On-chain handles ownership and verification; off-chain handles computation and player experience.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Likelihood/Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

Commit Hash

e6d2b2a64393650c668a0aa9f63f8218abda6c1c

Scope

Id	Files in scope
1	instructions/token/jail_token.rs
2	instructions/token/mod.rs
3	instructions/breeding/vrf.rs
4	instructions/breeding/pickaxe.rs
5	instructions/breeding/mod.rs
6	instructions/marketplace/mod.rs
7	instructions/marketplace/marketplace.rs

Id	Files in scope
8	instructions/admin/mod.rs
9	instructions/admin/burn.rs
10	instructions/admin/admin.rs
11	instructions/exploration/start.rs
12	instructions/exploration/mod.rs
13	instructions/exploration/loadout.rs
14	instructions/exploration/rewards.rs
15	instructions/mod.rs
16	instructions/equipment/pickaxe_level.rs
17	instructions/equipment/gem_inventory.rs
18	instructions/equipment/initialize.rs
19	instructions/equipment/pickaxe_repair.rs
20	instructions/equipment/mod.rs
21	instructions/equipment/gem_level.rs
22	instructions/equipment/pickaxe_inventory.rs
23	instructions/equipment/gems.rs
24	instructions/boxes/mystery_rewards.rs
25	instructions/boxes/mod.rs
26	instructions/boxes/pickaxe_boxes.rs
27	utils/mod.rs
28	utils/vrf_helpers.rs
29	utils/signature.rs
30	utils/shared.rs
31	constants.rs
32	heap.rs
33	lib.rs
34	state.rs
35	errors.rs

Roles

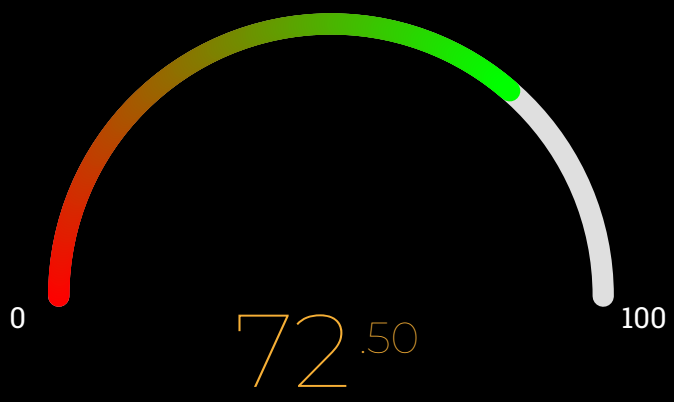
Id	Roles
1	Admin
2	Operator
3	User

Executive Summary

Issues Found

Severity	Count	Description
High	2	Critical vulnerabilities
Medium	7	Significant risks
Low	14	Minor issues with low impact
Informational	5	Best practices or suggestions
Gas	1	Optimization opportunities

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 72.50 / 100.00. This reflects a moderate security posture with some room for improvement.

Findings

High

High 01 Listing Seed Collision

Location

marketplace.rs

Description

The listing PDA is derived using seeds `[b"listing", seller.key().as_ref(), item_id.to_le_bytes().as_ref()]` across all three marketplace operations: `list_item`, `buy_item`, and `cancel_listing`. Critically, the `item_type` field is omitted from the seed derivation, even though listings can contain three distinct item types: `Pickaxe`, `Gem`, and `PickaxeBox`.

Item IDs can naturally overlap. For example, both a pickaxe and a gem could have ID 42. When a player attempts to list items of different types with colliding IDs, the PDA derivation produces the same address for both, causing a collision.

Seed collision anatomy:

```
Pickaxe ID 42:
  Seeds: [b"listing", seller_pubkey, 42.to_le_bytes()]
  PDA: Address_X

Gem ID 42:
  Seeds: [b"listing", seller_pubkey, 42.to_le_bytes()]
  PDA: Address_X  ← Same address!
```

When the player attempts to list the gem after listing the pickaxe:

- `list_item` tries to initialize a listing PDA with the same address

- The `init` constraint fails because the account already exists (it holds the pickaxe listing)
- The transaction reverts
- The player cannot list the gem unless they first cancel the pickaxe listing

This creates a functional denial-of-service for players and marketplace sellers: they cannot simultaneously list items of different types if those items share the same ID.

Proof of Concept

Scenario: Player unable to list both a pickaxe and gem with the same ID

1. Player owns two items with the same ID:
    - Pickaxe with ID = 42
    - Gem with ID = 42
    - (ID 42 is valid for both types in the game's ID assignment scheme)
2. Player lists the pickaxe:

```
list_item(
    item_id: 42,
    item_type: Pickaxe,
    price: 10_000_000
)
```

    - Seeds computed: `[b"listing", player_pubkey, [42, 0, 0, 0, 0, 0, 0, 0]]`
    - PDA derived: Address_42_Pickaxe
    - Listing created successfully
    - Pickaxe moved into the Listing PDA
3. Player lists the gem (same ID, different type):

```
list_item(
    item_id: 42,
    item_type: Gem,
    price: 5_000_000
)
```

    - Seeds computed: `[b"listing", player_pubkey, [42, 0, 0, 0, 0, 0, 0, 0]]`

- PDA derived: Address_42_Pickaxe <- Same as pickaxe listing!
- Anchor constraint: `init` tries to create a new account at this address
- Error: Account already exists (occupied by pickaxe listing)
- Transaction fails; gem cannot be listed

4. Player's options:
    - Cancel the pickaxe listing, then list the gem
    - Or accept that only one item with ID 42 can be listed at a time
    - Result: Reduced marketplace usability and forced delistings

Recommendation

Include the `item_type` in the Listing PDA seed derivation to ensure unique PDAs for each item type and ID pair per seller. This eliminates collisions and allows players to list multiple items with the same ID under different types simultaneously.

Update the account definitions:

```
#[derive(Accounts)]
#[instruction(item_id: u32, price: u64, item_type: MarketplaceItemType
    )]
pub struct ListItem<'info> {
    #[account(
        init,
        payer = seller,
        space = Listing::SPACE,
        seeds = [
            b"listing",
            seller.key().as_ref(),
            &(item_type as u8).to_le_bytes(),
            item_id.to_le_bytes().as_ref()
        ],
        bump
    )]
    pub listing: Account<'info, Listing>,
    // ... rest of struct ...
}

#[derive(Accounts)]
#[instruction(item_id: u32)]
pub struct BuyItem<'info> {
    #[account(
```

```
        mut,
        seeds = [
            b"listing",
            listing.seller.as_ref(),
            &(listing.item_type as u8).to_le_bytes(),
            item_id.to_le_bytes().as_ref()
        ],
        bump = listing.bump,
        constraint = listing.is_active @ GameError::ListingInactive,
        close = seller
    )]
    pub listing: Account<'info, Listing>,
    // ... rest of struct ...
}

#[derive(Accounts)]
#[instruction(item_id: u32)]
pub struct CancelListing<'info> {
    #[account(
        mut,
        seeds = [
            b"listing",
            seller.key().as_ref(),
            &(listing.item_type as u8).to_le_bytes(),
            item_id.to_le_bytes().as_ref()
        ],
        bump = listing.bump,
        constraint = listing.seller == seller.key() @ GameError::
    UnauthorizedSeller,
        close = seller
    )]
    pub listing: Account<'info, Listing>,
    // ... rest of struct ...
}
```

The fix is minimal and maintains all existing transaction logic while eliminating the collision vulnerability.

Status

Fixed

High 02 Listing Items Will Always Fail Due To Mutating A Read-only Account

Location

marketplace.rs

Description

In the `ListItem` accounts struct, the `marketplace_config` field is declared without the `mut` keyword:

```
#[derive(Accounts)]
#[instruction(item_id: u32, price: u64)]
pub struct ListItem<'info> {
    // ... other fields ...
    #[account(
        seeds = [b"marketplace_config"],
        bump,
        constraint = marketplace_config.is_active @ GameError::MarketplaceInactive
    )]
    pub marketplace_config: Account<'info, MarketplaceConfig>,
    // ...
}
```

However, in the `list_item` handler function, the code attempts to mutate `marketplace_config` by incrementing `total_listings`:

```
pub fn list_item(
    ctx: Context<ListItem>,
    // ... parameters ...
) -> Result<()> {
    // ... validation logic ...

    // Update total_listings for marketplace stats
    let marketplace_config = &mut ctx.accounts.marketplace_config;
    marketplace_config.total_listings = marketplace_config.total_listings.checked_add(1).unwrap();

    // ... emit event ...
    Ok(())
}
```

In Anchor, attempting to mutate an account that is not declared as `mut` in the `Accounts` struct causes a runtime error. The Anchor framework enforces that:

1. Read-only accounts must not be mutated
2. Mutable accounts must be explicitly marked with the `#[account(mut)]` or `mut` field attribute

Since `marketplace_config` is read-only (not marked as `mut`), the constraint checking phase passes (the account is loaded successfully), but the mutation attempt fails at runtime with an error indicating the account is read-only.

Result: The `list_item` function always fails when attempting to execute, preventing any listings from being created.

Proof of Concept

Scenario: Player attempts to list an item; transaction fails

1. Player calls `list_item` with valid parameters:

```
list_item(
    item_id: 42,
    price: 10_000_000,
    item_type: Pickaxe,
    floor_data: { ... },
    floor_signature: [ ... ]
)
```

2. Anchor account validation phase (succeeds):
    - `listing` PDA is initialized
    - `seller_profile` is loaded and marked mutable
    - `seller_pickaxes` is loaded and marked mutable
    - `marketplace_config` is loaded (read-only)
    - All constraints pass
3. Handler executes:
    - Item is captured from inventory
    - Listing PDA is populated
    - Handler reaches the mutation line:

```
let marketplace_config = &mut ctx.accounts.marketplace_config;
marketplace_config.total_listings = marketplace_config.total_listings.
    checked_add(1).unwrap();
```

4. Runtime error occurs:
    - Anchor detects that `marketplace_config` (marked read-only in the struct) is being mutated
    - Transaction fails; listing is not created

Recommendation

Mark the `marketplace_config` field as mutable in the `ListItem` accounts struct by adding the `mut` keyword:

```
#[derive(Accounts)]
#[instruction(item_id: u32, price: u64)]
pub struct ListItem<'info> {
    #[account(
        init,
        payer = seller,
        space = Listing::SPACE,
        seeds = [
            b"listing",
            seller.key().as_ref(),
            item_id.to_le_bytes().as_ref()
        ],
        bump
    )]
    pub listing: Account<'info, Listing>,
    #[account(
        mut,
        seeds = [b"player_profile", seller.key().as_ref()],
        bump
    )]
    pub seller_profile: Account<'info, PlayerProfile>,
    #[account(
        mut,
        seeds = [b"player_pickaxes", seller.key().as_ref()],
        bump = seller_pickaxes.bump,
    )]
    pub seller_pickaxes: Account<'info, PlayerPickaxeInventory>,
    #[account(
        mut,
        seeds = [b"marketplace_config"],
        bump,
```

```
32          constraint = marketplace_config.is_active @ GameError::
    MarketplaceInactive
33      )]
34      pub marketplace_config: Account<'info, MarketplaceConfig>,
35
36      /// Game config (pause checks)
37      #[account(
38          seeds = [crate::constants::CONFIG_SEED],
39          bump = game_config.bump
40      )]
41      pub game_config: Account<'info, GameConfig>,
42
43      #[account(mut)]
44      pub seller: Signer<'info>,
45      pub system_program: Program<'info, System>,
46      /// CHECK: Instructions sysvar for ed25519 verification (validated
        in handler)
47      #[account(address = anchor_lang::solana_program::sysvar::
    instructions::id())]
48      pub instructions_sysvar: AccountInfo<'info>,
49  }
```

Status

Fixed

Medium

Mid 01 No Config Initialization Protection

Location

admin.rs:325

Description

The `initialize_config` instruction has a critical authorization flaw: any signer can invoke this instruction and become the `config.admin`, the highest authority in the system as long as it was not invoked beforehand.

The `InitializeConfig` struct only requires that the `admin` account is a `Signer` and the payer for the `GameConfig` PDA creation. There is no constraint verifying that the signing account matches a hardcoded or expected authority. This means:

1. Any entity can call `initialize_config`
2. They will become the `config.admin` with full control over the protocol
3. Once set as admin, they can:
    - Pause/unpause the game (including emergency pause)
    - Grant/revoke player access
    - Create pickaxes, gems, and boxes
    - Manage operators
    - Update signature authorities

This is a privilege escalation vulnerability. The lack of authority verification violates the principle of least privilege and makes the system vulnerable to malicious initialization.

Recommendation

Add an explicit authority constraint to the `InitializeConfig` context that verifies the signer matches an expected hardcoded authority. This ensures only the intended party can initialize the game configuration:

```
/// Context for initializing game settings.
/// Run once to set global parameters.
#[derive(Accounts)]
pub struct InitializeConfig<'info> {
    #[account(
        mut,
        constraint = admin.key() == crate::constants::authorities::EXPECTED_INITIALIZER
            @ GameError::UnauthorizedInitializer
    )]
    pub admin: Signer<'info>,

    /// Game-wide config PDA storing authorities, pause flags, and allowlist settings.
    #[account(
        init,
        payer = admin,
        space = GameConfig::LEN,
        seeds = [crate::constants::CONFIG_SEED],
```

```
        bump
    )]
    pub config: Account<'info, GameConfig>,

    pub system_program: Program<'info, System>,
}
```

Status

Fixed

Mid 02 Wrong Deserialization Of `burn_pickaxes` Instruction

Location

burn.rs:149

Description

The `burn_pickaxes` function is designed to burn multiple pickaxes in a single transaction by accepting a `Vec<u32>` of pickaxe IDs. However, it reuses the `BurnPickaxe` context, which is constrained to accept only a single `pickaxe_id` as an instruction argument:

```
#[derive(Accounts)]
#[instruction(target_player: Pubkey, pickaxe_id: u32)]  // Only single pickaxe_id
pub struct BurnPickaxe<'info> {
    // ... accounts ...
}

pub fn burn_pickaxes(
    ctx: Context<BurnPickaxe>,           // Wrong context!
    target_player: Pubkey,
    pickaxe_ids: Vec<u32>,               // Multiple IDs
) -> Result<()> {
    // ...
}
```

This creates a type mismatch between the instruction context and the function signature. This will lead to instruction deserialization failure. Anchor's instruction parsing framework expects the instruction data to contain `target_player: Pubkey` and a single `pickaxe_id: u32`. When `burn_pickaxes` passes a `Vec<u32>`, the serialization format won't match, causing deserialization to fail.

Recommendation

Create a dedicated context for bulk pickaxe burning that accurately reflects the function's requirements:

```
/// Context for admin bulk pickaxe burns (primarily for sync fixes or test cleanup).
#[derive(Accounts)]
#[instruction(target_player: Pubkey, pickaxe_ids: Vec<u32>)]
pub struct BurnPickaxes<'info> {
    /// Admin with burn authority
    #[account(mut)]
    pub admin: Signer<'info>,

    /// Game config (verifies admin authority)
    #[account(
        seeds = [crate::constants::CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GameError::InvalidAdminAuthority
    )]
    pub config: Account<'info, GameConfig>,

    /// Target player's profile
    #[account(
        mut,
        seeds = [b"player_profile", target_player.as_ref()],
        bump = player_profile.bump,
    )]
    pub player_profile: Account<'info, PlayerProfile>,

    /// Target player's pickaxe inventory
    #[account(
        mut,
        seeds = [b"player_pickaxes", target_player.as_ref()],
        bump = player_pickaxes.bump,
    )]
    pub player_pickaxes: Account<'info, PlayerPickaxeInventory>,

```

```
    pub system_program: Program<'info, System>,
}

/// Burn multiple pickaxes as an admin.
///
/// - Removes multiple pickaxes from the player's inventory in a single transaction
/// - Deduplicates and sorts IDs before processing to prevent redundant work
/// - Intended for batch test cleanup and resolving bad data
/// - Use cautiously in production; removal is irreversible
pub fn burn_pickaxes(
    ctx: Context<BurnPickaxes>,
    target_player: Pubkey,
    pickaxe_ids: Vec<u32>,
) -> Result<()> {
    if pickaxe_ids.is_empty() {
        return Ok(());
    }

    let mut unique_ids = pickaxe_ids;
    unique_ids.sort_unstable();
    unique_ids.dedup();

    for pickaxe_id in unique_ids {
        burn_pickaxe_internal(
            &mut ctx.accounts.player_profile,
            &mut ctx.accounts.player_pickaxes,
            pickaxe_id,
            target_player,
            ctx.accounts.admin.key(),
        )?;
    }

    Ok(())
}
```

This approach:

- Creates proper alignment between the context type signature and function signature
- Allows Anchor's instruction framework to correctly deserialize the `Vec<u32>` argument
- Prevents runtime deserialization failures

Status

Fixed

Mid 03 Admin Burn Of Main Pickaxe During Active Exploration Can Leave Player Stuck In `is_exploring = true` State

Location

burn.rs:175

Description

When an admin burns a pickaxe that is currently equipped as the main pickaxe while the player is in an active exploration session (`is_exploring = true`), the pickaxe is correctly unequipped from the loadout, but the `is_exploring` flag is never cleared.

Exploration sessions are only ended via `claim_exploration_rewards` (player-initiated with a backend-signed payload that deducts HP). If the main pickaxe no longer exists, the backend is likely to reject the claim (no valid pickaxe to apply HP damage to), leaving the player permanently stuck in the exploring state.

Consequences:

- Player cannot fully modify their loadout (main slot locked).
- Cannot start new sessions reliably.
- Requires manual off-chain intervention (e.g., special backend claim or admin fix) to resolve.

While admin burns are intended for rare cleanup/sync fixes, this creates a state inconsistency where a player can become locked out of normal gameplay progression.

Proof of Concept

1. Player starts exploration (`start_exploration`) -> `is_exploring = true`, main pickaxe = ID `123`.

2. Admin calls `burn_pickaxe` (or bulk) on ID `123`.-
    - Pickaxe removed from inventory.
    - Loadout main slot cleared (`main_pickaxe_id = None`).
    - `is_exploring` remains `true`.
3. Player attempts `claim_exploration_rewards` -> backend rejects (no main pickaxe for HP deduct).
4. Player stuck — cannot end session or equip new main pickaxe normally.

Recommendation

In `burn_pickaxe_internal` (shared by all admin burn paths), add a safety check to force-end exploration if the burned pickaxe was the main one:

```
if player_profile.is_exploring
    && player_profile.exploration_loadout.main_pickaxe_id == Some(
    pickaxe_id)
{
    // Prevent stuck state by force-ending the session
    player_profile.is_exploring = false;

    // Optional: emit an event for monitoring
    emit!(ExplorationForceEndedByAdmin {
        player: player_profile.owner,
        pickaxe_id,
    });
}
```

This ensures the player can immediately recover and continue gameplay. Alternatively, document strictly: “Do not admin-burn the main pickaxe during active exploration.”

Status

Fixed

Mid 04 Repair Pickaxe Signed Payload Lacks Current HP — Risk of Over/Underpayment Due To Concurrent State Changes

Location

pickaxe_repair.rs:109

Description

The `repair_pickaxe` instruction allows a player to fully heal a pickaxe by burning JAIL, validated by a backend-signed payload.

The signed message includes:

- Player pubkey
- Pickaxe ID
- New HP (always `MAX_HP`)
- JAIL burn amount
- Nonce
- Expiry

It does NOT include the previous/current HP.

This creates a potential off-chain/on-chain desync risk:

- Backend calculates burn cost based on current HP (damage amount = MAX_HP - current_hp).
- If on-chain HP changes between cost calculation and transaction submission (e.g., concurrent exploration claim deducts HP), the signed cost becomes invalid for the actual damage.
- Player either:
  - Pays too much JAIL (for old higher damage).
  - Or backend rejects re-sign -> player stuck with damaged pickaxe.

While not a direct exploit (player can't force lower cost), it leads to poor user experience — frequent failed repairs requiring new signatures.

Similar risk exists in other signed actions if state changes between signing and execution.

Proof of Concept

1. Player has pickaxe with HP = 50. Backend signs repair cost for 50 damage (e.g., 10 JAIL).
2. Before player submits tx, concurrent exploration claim runs -> HP drops to 30.
3. Player submits original signed repair (for 50 damage).
4. On-chain: Heals to 100, burns 10 JAIL -> player overpays (should have been ~13 JAIL for 70 damage).
5. Or backend refuses to re-sign -> player waits.

Recommendation

Include the expected current HP in the signed message:

```
let message = format!(
    "repair_pickaxe_{}_{}_{}_{}_{}_{}_{}",
    player.key(),
    pickaxe_id,
    current_hp,      // ← Add this
    new_hp,
    jail_burn_amount,
    nonce,
    expires_at
);
```

Then in handler:

```
let current_hp = pickaxe.hp;
require!(expected_current_hp == current_hp, GameError::
    StaleRepairPayload);
```

This ensures:

- Cost matches actual damage at execution time.
- Prevents over/underpayment due to concurrent state changes.

Status

Fixed

Mid 05 Player's Box Is Burned Immediately On Request Which Can Lead To Permanent Loss

Location

pickaxe_boxes.rs

Description

The `request_pickaxe_box_opening` function has an ordering issue where it permanently removes the pickaxe box from the player's inventory before successfully completing the VRF randomness request:

```
pub fn request_pickaxe_box_opening(
    ctx: Context<RequestPickaxeBoxOpen>,
    attempt_id: u64,
    pickaxe_box_id: u32,
) -> Result<()> {
    // ... validation ...

    let player_profile = &mut ctx.accounts.player_profile;
    let box_index = player_profile.pickaxe_box_index(pickaxe_box_id)?;

    // Remove (burn) the box BEFORE VRF request succeeds
    player_profile.pickaxe_boxes.remove(box_index);

    let clock = Clock::get()?;
    let attempt = &mut ctx.accounts.box_attempt;
    // ... initialize attempt ...

    // Request randomness AFTER box is already deleted
    request_vrf_randomness(
        &ctx.accounts.player.key(),
        &ctx.accounts.oracle_queue.key(),
        // ... other params ...
    )?;

    emit!(PickaxeBoxOpenRequested { /* ... */ });

    Ok(())
}
```

This creates a permanent loss of assets scenario:

1. Oracle/Network Issues:

The MagicBlock VRF system may accept the request but never fulfil it (network outage, oracle failure, etc.). The box is permanently lost and cannot be recovered since:

- It's already removed from `player_profile.pickaxe_boxes`
- The `PickaxeBoxOpenAttempt` PDA exists but is never finalized
- No recovery mechanism restores the box

2. Operator Never Resolves:

Even if randomness is provided via `fulfill_pickaxe_box_randomness`, an operator could choose not to call `resolve_pickaxe_box_opening`. The player's box is burned with no rewards distributed, and there's no timeout or recovery path.

This violates the principle of atomicity: either the entire box-opening flow succeeds (box burned, rewards granted), or nothing should change. Currently, the box is destroyed regardless of whether the VRF flow completes.

Recommendation

Implement an attempt state machine that only burns the box after confirming successful VRF resolution, with a timeout mechanism for recovery:

Option 1: Defer box removal until resolution (safest)

Don't remove the box until the VRF flow is complete.

Option 2: Add timeout-based recovery mechanism

If you must remove the box early, allow players to recover it after a timeout.

Status

Acknowledged

Response: If we defer the reflection of VRF results, the risk of double execution by users will spike dramatically, and discrepancies between on-chain state and off-chain data will become more likely.

Mid 06 Irreversible Token Burns And Cooldowns On Unfulfilled Or Stalled Requests

Location

pickaxe.rs:317

Description

In the `request_pickaxe_breeding` function, irreversible state changes and resource consumption occur before the VRF randomness is guaranteed to be fulfilled or resolved:

1. JAIL tokens are burned immediately via `burn_jail_if_needed()`, permanently removing the player's cost from circulation;
2. Pickaxe cooldowns are updated via `enforce_pickaxe_cooldown()`, placing both parent pickaxes on a breeding cooldown;
3. Breeding counts are incremented via `update_parent_usage()`, moving each parent toward a per-pickaxe breeding limit;
4. Nonce is recorded via `check_and_add_nonce()`, consuming the nonce to prevent replay attacks;
5. BreedingAttempt PDA is initialized, consuming rent paid by the player.

All of these actions are committed before the VRF randomness request is sent. If the VRF request subsequently fails (due to oracle queue congestion, network issues, or other operational problems), or if an operator simply never calls `resolve_pickaxe_breeding`, the player has:

- Lost JAIL tokens permanently (burned, cannot be recovered)
- Lost game-time through cooldowns (cannot breed these parents for the duration)
- Lost breeding count allocations (counts toward a per-pickaxe limit)
- Lost rent (BreedingAttempt PDA remains unresolved and unreclaimed)

There is no refund mechanism, timeout, or cancellation instruction to reverse these costs if the VRF workflow fails to complete. The player bears the full cost of a breeding attempt with zero guarantee of receiving an outcome (success or failure resolution).

Proof of Concept

Scenario: VRF request fails; player loses resources permanently

1. Player initiates breeding request:

```
request_pickaxe_breeding(
 parent1_id: 42,
 parent2_id: 73,
 cost: 1000 micro-JAIL
)
```

2. Within the same transaction, in order:
   - Cooldown check passes; `enforce_pickaxe_cooldown()` sets `parent1_cooldown.last_bred_at = now` and `parent2_cooldown.last_bred_at = now`
   - Breeding counts incremented: `parent1.breeding_count += 1`, `parent2.breeding_count += 1`
   - Nonce recorded: `player_profile.nonces` updated to include `cost_data.nonce`
   - JAIL burned: `burn_jail_if_needed()` invokes Token-2022 burn CPI -> 1000 JAIL units removed from player's ATA permanently
   - BreedingAttempt PDA initialized: rent-exempt lamports transferred from player's account
3. VRF randomness request issued:
   - `request_breeding_randomness()` is called
   - MagicBlock VRF queue is invoked
   - Oracle queue is congested or fails to process the request
   - VRF callback never receives a response, or response is never submitted on-chain
4. Consequences for the player:
   - JAIL burned: 1000 JAIL units are gone forever (no refund possible)
   - Cooldown active: Both parent pickaxes are locked for the next 24 hours
   - Breeding count consumed: Each parent's breeding count incremented by 1, moving toward a per-pickaxe limit (e.g., max 10 breedings)
   - No outcome: The BreedingAttempt never enters `RandomnessReady` status; no `BreedingRandomnessReady` event is emitted; no rewards can be claimed

- Account stranded: The BreedingAttempt PDA remains in `AwaitingRandomness` state indefinitely, with rent permanently locked (cannot be closed because the attempt is not resolved)
- No recourse: Player cannot cancel, refund, or timeout the attempt

Recommendation

Implement a timeout and cancellation mechanism to allow players to recover from VRF failures. This will allow players to cancel an unresolved breeding attempt and refund costs if the given timeout has elapsed.

Status

Acknowledged

Response: If we defer the reflection of VRF results, the risk of double execution by users will spike dramatically, and discrepancies between on-chain state and off-chain data will become more likely.

Mid 07 Pre-funding PDA Griefing Attack

Location

admin.rs:19

pickaxe_boxes.rs:54

pickaxe.rs:450

gem_level.rs:270

initialize.rs:29

initialize.rs:38

initialize.rs:85

initialize.rs:94

marketplace.rs:75

marketplace.rs:131

jail_token.rs:34

jail_token.rs:54

jail_token.rs:70

Description

This vulnerability, known as PDA Initialization Griefing or Pre-funding DoS on PDA Creation, occurs in Solana programs using Anchor when a program-derived address (PDA) is created with the standard `init` constraint.

The PDA address is deterministic and can be computed off-chain by anyone using the same seeds (e.g., `"listing"`, seller's pubkey, and `item_id`). An attacker can pre-fund this address by transferring a tiny amount of SOL (even 1 lamport) to it before the legitimate user attempts creation.

Anchor's `init` constraint relies on the System Program's `create_account` instruction, which requires the target address to have zero lamports and zero data length. Pre-funding violates this, causing the creation to fail permanently with an error (e.g., "account already in use").

As a result, the legitimate seller can never successfully initialize the `listing` PDA for that specific `seller` + `item_id` combination. This is a low-cost, permanent denial-of-service (DoS) attack targeting specific users or listings.

The `bump` seed does not prevent this, as it is part of the deterministic derivation process and yields the same predictable PDA address.

Recommendation

Replace the vulnerable `init` constraint with `init_if_needed` :

```
#[account(
    init_if_needed,
    payer = seller,
    space = Listing::SPACE,
    seeds = [
        b"listing",
        seller.key().as_ref(),
        item_id.to_le_bytes().as_ref()
    ],
    bump
)]
pub listing: Account<'info, Listing>,
```

- If the account does not exist -> creates it normally.

- If pre-funded (or already exists) -> skips creation, validates seeds/ownership, and loads the account.

If the account must be new then make the seed as unpredictable as possible. This way the chance of a malicious actor precomputing it goes very low.

Status

Fixed

Response: Both mitigations were implemented - some accounts use `init_if_needed` instead of `init` and some have a more unpredictable seed.

Low

Low 01 Rent Returned To Operator Closing Which Can Be Different To The One Opening The Player Account

Location

admin.rs:203

Description

The `RevokePlayerAccess` instruction uses the `close = operator` constraint to close the `PlayerAccessRecord` PDA and return its rent to the signer of the revocation transaction. However, the operator revoking a player's access may be different from the operator who originally granted access via `GrantPlayerAccess`.

This creates an asymmetry: the rent deposit for creating the `PlayerAccessRecord` was paid by one operator (the original granter), but the rent refund upon closure goes to a different operator (the one executing the revocation). This allows an operator to receive SOL compensation they did not originally fund, effectively transferring rent value across different parties without explicit accounting.

While not a critical security issue, this represents a minor accounting inconsistency and potential unintended value transfer in the allowlist management system.

Recommendation

To maintain consistency and proper accounting restrict revocation to the original granter. Modify the constraint to require that the revoking operator matches the one who granted access. Store the granting operator's public key in `PlayerAccessRecord`.

Status

Fixed

Low 02 Global Uniqueness Not Enforced Potentially Leading To Problems

Location

admin.rs:437

admin.rs:473

admin.rs:514

Description

The `create_pickaxe`, `create_gem` and `create_pickaxe_box` instructions enforce ID uniqueness only within a single player's inventory, not globally across all players in the game. The validation check:

```
require!(
    !player_pickaxes.pickaxes.iter().any(|p| p.id == pickaxe_id),
    GameError::PickaxeIdAlreadyExists
);
```

or

```
require!(
    !player_profile.gems.iter().any(|g| g.id == gem_id),
    GameError::GemIdAlreadyExists
);
```

or

```
require!(
    !player_profile.pickaxe_boxes.iter().any(|b| b.id == box_id),
    GameError::PickaxeBoxIdAlreadyExists
);
```

only prevent duplicate IDs within the specific account being modified. This means:

1. Multiple players can own items with identical IDs (e.g., Player A and Player B can both have pickaxe ID 1)
2. Off-chain systems expecting IDs to be globally unique may experience issues
3. If items are traded, transferred, or merged across players in future functionality, ID collisions could cause unexpected behavior
4. Analytics, indexing, and leaderboard systems that assume unique IDs may produce incorrect results

Recommendation

Enforce global uniqueness. Create a registry PDA to track all issued IDs of different items and check every time a new ID is assigned if it already exists.

This way all of the possible problems mentioned above will be avoided.

Status

Fixed

Low 03 Possible Replay Attack When More Than 32 Actions In The Same Time Window

Location

shared.rs:336

Description

The `check_and_add_nonce` function provides replay protection by tracking recent nonces in `player_profile.used_nonces` (a `Vec<u64>` capped at `MAX_NONCES = 32`). When the vector exceeds the limit, oldest nonces are evicted (FIFO).

This creates a replay window: once a nonce falls out of the 32-slot window (after the player performs 32+ new signed actions), it becomes reusable. An attacker (or malicious player) could replay an old signed payload with that nonce, provided the payload's `expires_at` has not passed.

While `expires_at` (checked separately) mitigates long-term replays, the nonce window alone is short (32 actions). If a player is highly active, old nonces could become reusable relatively quickly, weakening the short-term replay protection.

Recommendation

Ensure that a user cannot execute 32 or more operations within the same time window. Operations must not be removed from the tracking vector until they have expired. If an operation has not expired, it must not be replayable. To enforce this reliably, use a single monotonically increasing nonce per user, incremented with each operation.

Status

Fixed

Low 04 Gem Equip/Unequip Allowed On Sub Pickaxes During Active Exploration Session

Location

gems.rs:42

Description

During an active exploration session (`is_exploring = true`), the program correctly locks the main pickaxe — preventing any gem equip/unequip actions on it (`ExplorationEquipmentLocked` error).

However, sub pickaxes (the 9 passive slots) are not locked. Players can freely equip or unequip gems on sub pickaxes using `equip_gems`, `unequip_gems`, or `update_gems` while exploring.

This creates an inconsistency with the intended "equipment locked during exploration" behavior and with other lock checks (repair/level-up/burn only lock the main pickaxe).

Proof of Concept

1. Player starts exploration (`start_exploration`) -> `is_exploring = true`, with a sub pickaxe equipped in slot 0.
2. Player calls `equip_gems(pickaxe_id = sub_pickaxe_id, gem_ids = [ ... ])`.
    - The lock check only triggers if `pickaxe_id == main_pickaxe_id`.
    - Transaction succeeds -> gems equipped on sub pickaxe mid-exploration.
3. Same for `unequip_gems` or `update_gems`.

Expected behavior (consistent with main pickaxe): All equipped pickaxes (main + subs) should be locked.

Recommendation

Update the gem equipment instructions (`equip_gems`, `unequip_gems`, `update_gems`) to lock any equipped pickaxe (main or sub) during exploration:

```
if player_profile.is_exploring
    && player_profile.exploration_loadout.is_pickaxe_equipped(
    pickaxe_id)
{
    return err!(GameError::ExplorationEquipmentLocked);
}
```

This uses the existing `is_pickaxe_equipped` method, which already checks both main and all sub slots.

This change ensures full consistency: no gem modifications on any equipped pickaxe while exploring.

Status

Fixed

Response: Subpickaxes were removed.

Low 05 Selling A Pickaxe Resets The Breeding Cooldown

Location

marketplace.rs:378

Description

When a player lists a pickaxe on the marketplace (`list_item` with `item_type = Pickaxe`), the pickaxe is removed from the player's `PlayerPickaxeInventory` and locked in the `Listing` PDA escrow.

The associated `PickaxeBreedingCooldown` PDA (per-pickaxe cooldown timer) is not affected — it remains with its current `last_bred_at` timestamp.

However, when the listing is bought or canceled, the pickaxe is returned to the new owner (buyer) or original owner (cancel).

Because cooldown PDAs are derived with the owner's pubkey in the seeds:

```
seeds = [PICKAXE_BREEDING_COOLDOWN_SEED, owner.key(), pickaxe_id.
    to_le_bytes()]
```

- The buyer's cooldown PDA for this pickaxe ID is brand new (or `init_if_needed` with `last_bred_at = 0`).
- This effectively resets the breeding cooldown for the new owner.

This allows the buyer to immediately breed the purchased pickaxe, even if the seller had recently bred it (cooldown active).

Recommendation

To preserve cooldown across ownership changes, either:

1. Make cooldown global per pickaxe ID (remove owner from seeds):

```
seeds = [PICKAXE_BREEDING_COOLDOWN_SEED, pickaxe_id.to_le_bytes()]
```

- Single cooldown PDA per pickaxe ID (program-wide).
- Cooldown follows the pickaxe on trade.
- Requires migration for existing PDAs.

2. Or accept the reset as a feature: Trading “refreshes” cooldown -> encourages marketplace activity (buyers get immediate use).

Status

Fixed

Low 06 Missing `ensure_profile_capacity` Call

Location

pickaxe_repair.rs:57

marketplace.rs:371

pickaxe_level.rs:57

Description

In `repair_pickaxe`, `level_up_pickaxe`, `list_item`, `check_and_add_nonce` is called to add the nonce from the breeding cost signature, but `ensure_profile_capacity` is not called beforehand with `nonces(1)`.

This creates a minor edge-case risk: when the player’s `used_nonces` vector is exactly at `MAX_NONCES` (32), the `push(nonce)` temporarily increases length to 33 before `drain` reduces it back to 32. Rust Vec reallocs (typically doubles capacity) on push when full, requiring extra account space.

If the `PlayerProfile` account is not pre-resized for this temporary growth, the transaction can fail with “account data too small”.

Recommendation

Add pre-resize before `check_and_add_nonce` in all single-nonce instructions:

```
ensure_profile_capacity(
    &mut ctx.accounts.player_profile,
    &ctx.accounts.payer,  // player pays
    &ctx.accounts.system_program,
    CapacityRequest::nonces(1),
)?;
```

Status

Fixed

Low 07 Operators Have Unilateral Control Over Reward Creation

Location

mystery_rewards.rs

Description

The `open_mystery_box` function relies entirely on operator-provided reward data without implementing per-box or per-reward limits on the amounts that can be minted or added. While operators are whitelisted (verified via `game_config.operators.contains(&authority.key())`), a malicious or compromised operator could abuse this trust to:

1. Mint excessive JAIL tokens: The `jail_amount: u64` field in each `MysteryBoxReward` has no upper bound validation. An operator could mint up to `u64::MAX` JAIL tokens per box, causing token inflation.
2. Add unlimited pickaxes/gems: The `pickaxes: Vec<ExplorationPickaxe>` and `gems: Vec<Gem>` vectors are only checked against per-player storage limits (e.g., `PlayerProfile::MAX_GEMS`), not against reasonable reward expectations per box.

3. Resource exhaustion: While storage limits exist, an operator could create boxes with extremely large gem/pickaxe counts that approach or hit these limits in a single transaction, preventing legitimate gameplay.

Current validation only checks:

- Batch is non-empty and <= 10 boxes
- Storage won't be exceeded

There is no validation of the intrinsic reward values, such as:

- Minimum/maximum JAIL per box
- Reasonable gem/pickaxe count per box
- Total JAIL minted per transaction
- Rate limiting on operator actions

This is particularly concerning because operators have unilateral control over reward creation.

Recommendation

Implement per-reward validation limits to constrain what operators can distribute. This can be done either on-chain or enforced off-chain with on-chain verification:

Option 1: Hard on-chain limits (conservative)

Add explicit maximums for each reward component:

```
/// Validate individual reward amounts are within acceptable bounds
fn validate_individual_reward(reward: &MysteryBoxReward) -> Result<()>
    {
    // Define reasonable maximums
    const MAX_JAIL_PER_BOX: u64 = 1_000_000; // Adjust based on game
    economics
    const MAX_PICKAXES_PER_BOX: usize = 5;
    const MAX_GEMS_PER_BOX: usize = 10;

    require!(
        reward.jail_amount ⩽ MAX_JAIL_PER_BOX,
        GameError::ExcessiveJailReward
    );
    require!(
        reward.pickaxes.len() ⩽ MAX_PICKAXES_PER_BOX,
        GameError::ExcessivePickaxeReward
    );
```

```
    require!(
        reward.gems.len() <= MAX_GEMS_PER_BOX,
        GameError::ExcessiveGemReward
    );

    Ok(())
}

pub fn open_mystery_box(
    ctx: Context<OpenMysteryBox>,
    reward_batch: MysteryBoxRewardBatch,
) -> Result<()> {
    validate_player_context!(ctx, crate::constants::pause_flags::
    OPEN_MYSTERY_BOX);

    let rewards = reward_batch.rewards;
    validate_reward_batch(&rewards)?;

    // Validate each reward individually
    for reward in &rewards {
        validate_individual_reward(reward)?;
    }

    let reward_count = rewards.len();
    let (total_new_pickaxes, total_new_gems) = aggregate_reward_totals
    (&rewards);

    // ... rest of function
}
```

Option 2: Off-chain signed limits with on-chain verification

Have operators sign a manifest of expected reward amounts, verified on-chain:

```
/// Off-chain signed reward manifest
#[derive(AnchorSerialize, AnchorDeserialize, Clone, Debug)]
pub struct SignedRewardManifest {
    pub max_jail_total: u64,
    pub max_pickaxes_total: usize,
    pub max_gems_total: usize,
    pub manifest_nonce: u64,
    pub signature: [u8; 64], // Ed25519 signature
}

pub fn open_mystery_box(
    ctx: Context<OpenMysteryBox>,
    reward_batch: MysteryBoxRewardBatch,
    manifest: SignedRewardManifest,
```

```
) -> Result<()> {
    validate_player_context!(ctx, crate::constants::pause_flags::OPEN_MYSTERY_BOX);

    let rewards = reward_batch.rewards;
    validate_reward_batch(&rewards)?;

    // Verify manifest was signed by the operator
    verify_manifest_signature(&ctx.accounts.authority, &manifest)?;

    let (total_pickaxes, total_gems) = aggregate_reward_totals(&rewards);
    let total_jail: u64 = rewards.iter().map(|r| r.jail_amount).sum();

    // Enforce manifest limits
    require!(
        total_jail <= manifest.max_jail_total,
        GameError::ExceedsManifestJailLimit
    );
    require!(
        total_pickaxes <= manifest.max_pickaxes_total,
        GameError::ExceedsManifestPickaxeLimit
    );
    require!(
        total_gems <= manifest.max_gems_total,
        GameError::ExceedsManifestGemLimit
    );

    // ... rest of function
}
```

Status

Fixed

Response: Added admin-controlled cap for mystery box JAIL rewards. Users should trust operators.

Low 08 Missing `box_id` Validation Can Lead To Reusable Or Arbitrary IDs

Location

mystery_rewards.rs

Description

The `apply_mystery_box_reward` function accepts a `box_id` parameter but performs no validation that this ID corresponds to an actual, valid mystery box type defined in the game. The `box_id` is only used for emitting an event:

```
fn apply_mystery_box_reward<'info>(
    accounts: &mut OpenMysteryBox<'info>,
    reward: MysteryBoxReward,
    mint_authority_bump: u8,
) -> Result<()> {
    let MysteryBoxReward {
        box_id,  // No validation against known box types
        jail_amount,
        pickaxes,
        gems,
        nonce,
    } = reward;

    // ... processing ...

    emit!(MysteryBoxOpened {
        player: accounts.player.key(),
        operator: accounts.authority.key(),
        box_id,  // Arbitrary value can be emitted
        jail_amount,
        pickaxe_ids,
        gem_ids,
        nonce,
    });
}
```

This creates two distinct issues:

1. Arbitrary box IDs: Operators can pass any `u32` value as the `box_id`, including non-existent or invalid box types. This provides no guarantee that the reward corresponds to an actual box earned during exploration.
2. Reusable box IDs across transactions: The only replay protection in the system is the nonce-based prevention via `check_and_add_nonce`. This means:
    - Operator can claim to open box ID `1` with nonce `100` -> accepted
    - Operator can claim to open box ID `1` with nonce `101` -> accepted (different nonce)
    - Operator can claim to open box ID `1` with nonce `102` -> accepted (different nonce)

The same `box_id` can be "opened" unlimited times as long as each use has a unique nonce.

This undermines the intent of mystery boxes as earned rewards. Without `box_id` validation, there's no guarantee that:

- The box was actually earned by the player
- The box type is valid for the game's reward system
- The rewards align with the box's intended loot table

Recommendation

Implement `box_id` validation by maintaining an on-chain registry of valid mystery box types and optionally tracking opened boxes per player to prevent reuse. Validate `box_id` against a whitelist and track opened boxes to prevent reuse of the same physical box.

Status

Acknowledged

Response: Added incident response documentation for mystery box ID mismatches.

Low 09 `total_volume` In MarketplaceConfig Can Overflow On Extreme Trading Volume Or Zero-Fee Abuse

Location

marketplace.rs:548

Description

The `MarketplaceConfig` account tracks cumulative trading volume in SOL using a `total_volume: u64` field (lamports precision).

This field is incremented on every successful sale:

```
marketplace_config_mut.total_volume = marketplace_config_mut
    .total_volume
    .checked_add(sale_price)
    .unwrap();
```

A `u64` can hold up to ~18.4 quintillion lamports (~18.4 million SOL at current precision).

While reaching this limit would require astronomical trading volume (far beyond realistic game economics), two scenarios make overflow possible:

1. Long-term accumulation: Over many years/decades with high activity, volume could theoretically exceed `u64::MAX`.
2. Zero-fee abuse: The marketplace authority can set `fee_rate = 0`. An attacker (or colluding parties) could then perform millions of tiny trades (even 1 lamport) in a loop to deliberately overflow `total_volume`. - Each trade adds to `total_volume` regardless of fee.
    - `checked_add( ... ).unwrap()` will panic on overflow -> transaction reverts, potentially DoS-ing the marketplace until fixed.

Although not an immediate funds-loss vulnerability, overflow leads to transaction failures on legitimate sales once the limit is reached, effectively breaking the marketplace.

Proof of Concept

1. Admin sets `fee_rate = 0`.
2. Two colluding wallets perform repeated buy/sell cycles with `price = 1 lamport`.
3. After ~18.4 quintillion such trades, `total_volume` overflows `u64::MAX`.
4. Next legitimate sale -> `checked_add` panics -> all buys fail permanently until manual intervention (new program version or off-chain reset).

Even without malice, extremely high long-term volume (e.g., billions of SOL traded) could hit the limit naturally.

Recommendation

Replace `total_volume: u64` with a type that cannot overflow in practice, or remove checked arithmetic:

Option 1 (Recommended — Simple & Safe):

```
pub total_volume: u128,  // 128-bit integer — effectively unlimited
    for SOL volume
```

Update increment:

```
marketplace_config_mut.total_volume = marketplace_config_mut
    .total_volume
    .saturating_add(sale_price as u128);
```

Option 2: Keep `u64` but use saturating add (cap at max):

```
marketplace_config_mut.total_volume = marketplace_config_mut
    .total_volume
    .saturating_add(sale_price);
```

- Prevents panic, volume caps at `u64::MAX` (still huge: ~18 million SOL).

Option 3: Remove `total_volume` entirely if only used for display (off-chain indexing can compute from events).

Status

Fixed

Low 10 Rewards Supplied By Operator Not Checked Against Stored VRF

Location

pickaxe_boxes.rs

Description

The `resolve_pickaxe_box_opening` function accepts a `PickaxeBoxRewards` struct containing pickaxe IDs that is supplied directly by the operator. While the function performs several validation checks:

1. Verifies `attempt.status == RandomnessReady` (confirming VRF randomness was fulfilled)

2. Validates the nonce via `check_and_add_nonce` (replay protection)
3. Checks that pickaxe IDs don't duplicate existing inventory items

It does not verify that the supplied `rewards.pickaxes` were actually derived from the VRF randomness stored.

This creates a trust assumption: the operator is expected to honestly compute the rewards based on the VRF randomness, but the program has no mechanism to validate this claim. An operator could arbitrarily award any pickaxes to any player, completely bypassing the randomness that was intended to determine loot fairly. The VRF request and callback process becomes cosmetic - the actual reward distribution is entirely operator-controlled.

Recommendation

Implement deterministic reward computation on-chain that derives the pickaxe rewards from `attempt.roll` (or `attempt.randomness`). This could involve:

1. Define a reward table mapping roll ranges to specific pickaxe IDs or types
2. Compute expected rewards in `resolve_pickaxe_box_opening` using the stored randomness
3. Compare computed vs. supplied rewards before accepting them, or reject mismatches

This ensures the VRF randomness actually determines outcomes and prevents operator manipulation of the loot system.

Status

Acknowledged

Response: Users need to trust operators.

Low 11 Operator Can Supply Arbitrary `awarded_box_ids` Ignoring VRF Randomness

Location

pickaxe.rs:317

Description

`resolve_pickaxe_breeding` allows an `operator` to pass an arbitrary `awarded_box_ids : Vec<u32>`. The function checks only:

- that the attempt's `status == RandomnessStatus::RandomnessReady`;
- that `primary_roll` and `secondary_roll` are present (they are read from the attempt but not used to validate award);
- uniqueness and storage limits for the box IDs.

There is no check that `awarded_box_ids` were derived from the VRF randomness before awarding boxes. The operator can therefore always mark success and allocate desirable box IDs.

Recommendation

Make the `awarded_box_ids` cryptographically derived from the VRF randomness and verify the derived IDs inside `resolve_pickaxe_breeding`. For example: derive box IDs using HMAC (randomness || attempt_id || player) or use deterministic RNG seeded with `attempt.randomness` and then generate n box IDs; compare those derived IDs to `awarded_box_ids` provided by the operator.

Status

Acknowledged

Response: Users need to trust operators.

Low 12 Missing Treasury Account Validation On Initialization

Location

marketplace.rs:306

Description

The `initialize_marketplace_config` function accepts a `treasury` account without any validation of its ownership or type. The treasury is marked as a `UncheckedAccount<'info>` with only a comment (`/// CHECK: Treasury address validated during setup`), but no actual validation is performed in the handler code.

In contrast, the `update_marketplace_treasury` function does perform validation:

```
require!(
    ctx.accounts.new_treasury.owner == &system_program::ID,
    GameError::InvalidTreasury
);
```

This inconsistency creates a vulnerability: during initialization, an attacker (or misconfigured admin) can register a treasury account owned by an arbitrary program - such as:

- A malicious attacker's program
- A PDA controlled by an attacker
- Any program-derived account

Once the treasury is set to a compromised account, all marketplace fees collected via `buy_item` are transferred to that address. Since the account is program-owned (not a simple system account), the attacker's program can intercept, redirect, or lock the funds, making them irretrievable by legitimate users or the game authority.

Impact chain:

1. Admin initializes marketplace with attacker-controlled treasury via `initialize_marketplace_config`
2. Players buy items on the marketplace
3. `buy_item` calculates fees and transfers them to the compromised treasury:

```
let fee_transfer_ix = anchor_lang::solana_program::system_instruction
    ::transfer(
    buyer.key,
    treasury.key,  // <- Attacker's program account
    fee_amount,
);
```

4. Funds are now under the attacker's program control and cannot be recovered

The vulnerability exists because the initialization function lacks the ownership check that the update function correctly implements, creating an asymmetry in validation rigor.

Recommendation

Add the same ownership validation to `initialize_marketplace_config` that exists in `update_marketplace_treasury`. This ensures the treasury is always a system-owned account (not program-owned) and cannot be hijacked.

Status

Fixed

Low 13 Faulty Marketplace Pause Functionality

Location

marketplace.rs

Description

The marketplace has an `is_active` flag in `MarketplaceConfig` that is initialized to `true` during `initialize_marketplace_config`, but there is no instruction to disable it. The codebase provides no way for the marketplace authority to pause or shut down the marketplace.

More critically, even if the `is_active` flag could be disabled (through a hypothetical future instruction), the `buy_item` function does not check this flag. The flag is only validated in `list_item`:

```
// In list_item:
#[account(
    seeds = [b"marketplace_config"],
    bump,
    constraint = marketplace_config.is_active @ GameError::
    MarketplaceInactive
```

```
)]
pub marketplace_config: Account<'info, MarketplaceConfig>,
```

However, `buy_item` loads the marketplace config without any `is_active` check:

```
// In buy_item:
#[account(
    mut,
    seeds = [b"marketplace_config"],
    bump = marketplace_config.bump
)]
pub marketplace_config: Account<'info, MarketplaceConfig>,
// <- No constraint on is_active
```

This creates an operational gap: sellers can be blocked from listing new items (if the marketplace were disabled), but existing listings can still be purchased indefinitely, allowing fees to continue accumulating and items to be traded even when the marketplace should be offline.

Recommendation

Implement marketplace disable/enable logic and enforce it consistently across all marketplace operations or remove it from the whole protocol's logic.

Status

Fixed

Response: Removed the unused is_active field from MarketplaceConfig struct. The marketplace pause control is already properly handled through the global pause flags system.

Low 14 Operators Can Decide Whether A Gem Level Up Attempt Is Successful Or Not By Themselves

Location

gem_level.rs

Description

In the gem level-up flow, the `resolve_gem_level_up` function accepts a `success: bool` parameter supplied entirely by the operator, with no on-chain validation that this outcome aligns with the VRF-generated roll stored in `gem_attempt.roll`.

The flow:

1. Player calls `request_gem_level_up` -> burns 3 gems + JAIL cost -> VRF request issued
2. `fulfill_gem_level_up_randomness` callback writes the roll to `gem_attempt.roll`
3. Operator calls `resolve_gem_level_up(success: bool, ...)` with an arbitrary boolean value
4. If `success = true`, a new leveled gem is awarded; if `false`, nothing happens

The `resolve_gem_level_up` function performs zero checks linking the operator-provided `success` parameter to the stored `gem_attempt.roll`. There is no validation such as:

```
// Example: NOT present in the code
if success {
    require!(gem_attempt.roll ⩾ SUCCESS_THRESHOLD, GameError::
    InvalidOutcome);
}
```

The code only validates:

- Attempt status is `RandomnessReady`
- Signature expiry
- Operator authorization
- If success: gem level increment (+1) and inventory constraints

But never: “Does the roll actually support this success outcome?”

Consequences:

1. Operator can arbitrarily award gems: Even if a player’s roll is very low, the operator can declare `success = true` and award a leveled gem
2. Or arbitrarily deny success: A player with a roll of 9999 (very high) can have `success = false` declared, denying them the reward they earned
3. Economically broken: The VRF randomness becomes purely cosmetic; the operator controls all outcomes

4. Unfair game balance: Operators can favor certain players, craft exploitable odds, or trade favorable outcomes for bribes

The system has the infrastructure for fair randomness (VRF rolls are correctly computed and stored) but deliberately ignores it during resolution, creating a trust gap where outcomes are determined by operator fiat, not randomness.

Recommendation

Establish a success threshold based on the gem type and level, then enforce it during resolution. This way outcomes are determined by randomness, not operator discretion. What is more, success thresholds are public and immutable.

Status

Acknowledged

Response: Users need to trust operators.

Informational

Info 01 Redundant Event Emitted Sometimes When Setting Player Enforcement

Location

admin.rs:365

Description

The `set_player_access_enforcement` function unconditionally emits a `PlayerAccessEnforcement` event every time it is called, regardless of whether the `enforce_player_allowlist` flag actually changes value.

If an admin calls this function with the same boolean value that is already stored in the config (e.g., calling with `enforce=true` when it's already `true`), the state does not change, yet an event is still emitted. This creates unnecessary noise in the

event log and could mislead off-chain systems or indexers into thinking a meaningful state transition occurred when the configuration remained identical.

This is inefficient and violates the principle that events should only be emitted when state-changing operations actually occur.

Recommendation

Compare the new value against the current stored value before updating and emitting the event. Only emit the `PlayerAccessEnforcementUpdated` event if the value actually changes:

```
pub fn set_player_access_enforcement(
    ctx: Context<SetPlayerAccessEnforcement>,
    enforce: bool,
) -> Result<()> {
    let config = &mut ctx.accounts.config;

    // Only update and emit if the value actually changes
    if config.enforce_player_allowlist ≠ enforce {
        config.enforce_player_allowlist = enforce;
        emit!(PlayerAccessEnforcementUpdated {
            admin: ctx.accounts.admin.key(),
            enforce
        });
    }

    Ok(())
}
```

This ensures that events are only emitted for actual state transitions, reducing log noise and providing accurate signal to off-chain listeners.

Status

Fixed

Info 02 Redundant Event Emitted When `global_pause` Value Is Not Actually Changed

Location

admin.rs:579

Description

The `toggle_global_pause` function unconditionally emits a `GlobalPauseToggled` event every time it is called, regardless of whether the `global_pause` flag actually changes value.

If an admin calls this function with the same boolean value that is already stored in the config (e.g., calling with `global_pause=true` when it's already `true`), the state does not change, yet an event is still emitted. This creates unnecessary noise in the event log and could mislead off-chain systems, indexers, or UI displays into thinking a meaningful state transition occurred when the configuration remained identical.

Recommendation

Compare the new value against the current stored value before updating and emitting the event. Only emit the `GlobalPauseToggled` event if the value actually changes:

```
pub fn toggle_global_pause(ctx: Context<TogglePause>, global_pause: bool) -> Result<()> {
    let config = &mut ctx.accounts.config;

    // Only update and emit if the value actually changes
    if config.global_pause ≠ global_pause {
        config.global_pause = global_pause;
        emit!(GlobalPauseToggled {
            admin: ctx.accounts.admin.key(),
            global_pause,
        });
    }

    Ok(())
}
```

This ensures that events are only emitted for actual state transitions, reducing log noise and providing accurate signal to off-chain listeners that the global pause state has genuinely changed.

Status

Fixed

Info 03 Redundant Timestamp Variable In Creation Functions

Location

admin.rs

Description

The `_current_time` variable is captured in three creation functions (`create_pickaxe`, `create_gem`, and `create_pickaxe_box`) but is never used:

```
let _current_time = Clock::get()?.unix_timestamp;
```

The underscore prefix indicates it's intentionally unused, but this creates code smell and uncertainty:

1. Dead code: The system call to retrieve the clock's Unix timestamp consumes gas/compute budget but provides no value
2. Unclear intent: It's ambiguous whether this is placeholder code for future functionality or accidentally leftover from development
3. Maintenance burden: Future developers may wonder if this should be implemented and waste time investigating

Recommendation

Option 1: Remove the unused variable (if timestamps are truly not needed)

Simply delete the line from all three functions to clean up the code:

```rust
pub fn create_pickaxe(
    ctx: Context<CreatePickaxe>,
    target_player: Pubkey,
    pickaxe_id: u32,
    level: u16,
) -> Result<()> {
    // Removed: let _current_time = Clock::get()?.unix_timestamp;

    let player_pickaxes = &mut ctx.accounts.player_pickaxes;
    // ... rest of function
}
```

Option 2: Add timestamp to events (if timestamps should be tracked)

If you want to preserve item creation timestamps for game mechanics, include the timestamp in the event structs and emit it:

```rust
/// Event for pickaxe creation by an admin.
#[event]
pub struct PickaxeCreated {
    pub admin: Pubkey,
    pub player: Pubkey,
    pub pickaxe_id: u32,
    pub level: u16,
    pub total_pickaxes: u16,
    pub created_at: i64,  // Add timestamp
}

pub fn create_pickaxe(
    ctx: Context<CreatePickaxe>,
    target_player: Pubkey,
    pickaxe_id: u32,
    level: u16,
) -> Result<()> {
    let current_time = Clock::get()?.unix_timestamp;

    let player_pickaxes = &mut ctx.accounts.player_pickaxes;

    check_pickaxe_inventory_limit(player_pickaxes, 1)?;

    require!(
        !player_pickaxes.pickaxes.iter().any(|p| p.id == pickaxe_id),
        GameError::PickaxeIdAlreadyExists
    );

    let new_pickaxe = ExplorationPickaxe::new(pickaxe_id, level)?;
    player_pickaxes.pickaxes.push(new_pickaxe);
```

```
31
32      emit!(PickaxeCreated {
33          admin: ctx.accounts.admin.key(),
34          player: target_player,
35          pickaxe_id,
36          level,
37          total_pickaxes: player_pickaxes.pickaxes.len() as u16,
38          created_at: current_time,  // Include timestamp
39      });
40
41      Ok(())
42  }
```

Status

Fixed

Response: Removed the unused timestamp variables.

Info 04 `validate_player_profile` Check Is Not Needed

Location

pickaxe_repair.rs:76

gem_inventory.rs:31

pickaxe.rs:116

pickaxe_inventory.rs:33

pickaxe_level.rs:74

reward.rs:166

Description

The `validate_player_profile!` macro is called in many player-facing instructions (e.g., `repair_pickaxe`, `level_up_pickaxe`, `burn_owned_pickaxe`, gem equipment, etc.) to enforce:

```
require!(player_profile.owner == player.key(), GameError::
    PlayerNotOwner);
```

However, every relevant `#[derive(Accounts)]` context already includes the exact same constraint:

```
#[account(
    ...,
    constraint = player_profile.owner == player.key() @ GameError::
    PlayerNotOwner
)]
pub player_profile: Account<'info, PlayerProfile>,
```

Anchor enforces this constraint during account validation (before the handler runs). If it fails, the transaction aborts early with `PlayerNotOwner`.

Recommendation

Remove the `validate_player_profile!` macro calls from all player-facing instructions where the context already includes the equivalent constraint.

This reduces code duplication and avoids redundant checks.

Status

Fixed

Info 05 Confusing Error Returned On Duplicate IDs

Location

pickaxe.rs:812

Description

In the `ensure_unique_ids` function, when a duplicate ID is detected in the input vector, the code returns `GameError::InvalidSignature`:

```rust
pub(crate) fn ensure_unique_ids(ids: &[u32]) -> Result<()> {
    let mut set = HashSet::with_capacity(ids.len());
    for id in ids {
        if !set.insert(id) {
            return err!(GameError::InvalidSignature);
        }
    }
    Ok(())
}
```

However, `InvalidSignature` is semantically incorrect for this error condition. According to the error definition:

```rust
#[msg("The provided signature is invalid")]
InvalidSignature = 6000,
```

This error is meant to indicate cryptographic signature validation failures, not duplicate ID detection. Using it here is misleading because:

1. Semantic mismatch: The error message talks about signatures, but the actual problem is duplicate IDs in a collection
2. Log confusion: A caller reading logs will misinterpret duplicate IDs as a signature validation failure
3. Poor debugging: It obscures the actual root cause when troubleshooting failures

The codebase already has appropriate, specific errors for this scenario:

```rust
#[msg("Pickaxe with this ID already exists")]
PickaxeIdAlreadyExists = 6005,

#[msg("Pickaxe box with this ID already exists")]
PickaxeBoxIdAlreadyExists = 6009,

#[msg("Gem with this ID already exists")]
GemIdAlreadyExists = 6013,
```

Recommendation

Define or reuse a generic duplicate ID error and return it from `ensure_unique_ids`. Either:

Option 1: Create a new generic error

```rust
#[error_code]
pub enum GameError {
    // ... existing errors ...

    #[msg("Duplicate ID detected in input")]
    DuplicateId = 6XXX,
}
```

Then update the function:

```rust
pub(crate) fn ensure_unique_ids(ids: &[u32]) -> Result<()> {
    let mut set = HashSet::with_capacity(ids.len());
    for id in ids {
        if !set.insert(id) {
            return err!(GameError::DuplicateId);
        }
    }
    Ok(())
}
```

Option 2: Make the function generic and accept an error parameter

```rust
pub(crate) fn ensure_unique_ids(ids: &[u32], error: GameError) ->
    Result<()> {
    let mut set = HashSet::with_capacity(ids.len());
    for id in ids {
        if !set.insert(id) {
            return err!(error);
        }
    }
    Ok(())
}
```

Then call it contextually:

```rust
ensure_unique_ids(&awarded_box_ids, GameError::
    PickaxeBoxIdAlreadyExists)?;
ensure_unique_ids(&gem_ids, GameError::GemIdAlreadyExists)?;
```

The first approach is cleaner and more maintainable.

Status

Fixed

Gas

Gas 01 `PickaxeBoxOpenAttempt` Account Is Left Unclosed With Its Rent Unclaimed

Location

pickaxe_boxes.rs

Description

In Solana, accounts consume rent - a proportional amount of SOL based on their data size, required to keep them alive on-chain. When an account is no longer needed, it should be closed to reclaim that rent and return it to the payer or a designated account.

In the `resolve_pickaxe_box_opening` function, the `box_attempt` account is created in `request_pickaxe_box_opening` (via the `init` constraint, paid for by the player) and is used to track the VRF randomness resolution workflow. However, after the attempt is finalized and marked as `Resolved`, the account is never closed.

The `finalize_box_attempt` helper function only updates the attempt's status and timestamps - it does nothing to reclaim the account's rent. This means every pickaxe box opening leaves behind a permanent `PickaxeBoxOpenAttempt` account on-chain, with its rent cost permanently locked and unrecoverable by the player.

Over time, as many players open pickaxe boxes, this creates accumulating "dead" accounts that:

- Consume blockchain state space
- Represent lost SOL (rent) for players who paid the initialization cost
- Increase the overall data bloat of the program

Recommendation

Close the `box_attempt` account at the end of `resolve_pickaxe_box_opening` by using Anchor's `close` constraint:

Modify the `box_attempt` field in the `ResolvePickaxeBoxOpen` struct to include the `close` attribute:

```rust
#[derive(Accounts)]
#[instruction(attempt_id: u64, rewards: PickaxeBoxRewards)]
pub struct ResolvePickaxeBoxOpen<'info> {
    #[account(mut)]
    pub operator: Signer<'info>,

    #[account(mut)]
    pub player: SystemAccount<'info>,

    #[account(
        mut,
        close = player,
        seeds = [PICKAXE_BOX_ATTEMPT_SEED, box_attempt.player.as_ref()
    , &attempt_id.to_le_bytes()],
        bump = box_attempt.bump
    )]
    pub box_attempt: Account<'info, PickaxeBoxOpenAttempt>,
    // ... existing fields ...
}
```

Closing the account prevents indefinite rent accumulation and reclaims blockchain resources.

Status

Fixed